SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA10 / CSA1005	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	**KARNAM HARSHITH**	**Reg. No.:**	**192511384**
Date:		**Duration:**	60 minutes

Code of Conduct

I **KARNAM HARSHITH 192511384** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: _____

Annexure A – Architecture Design Assignment

Instructions to Students:

Each student will be assigned an individual software project problem statement. Based on the assigned problem, analyze the system requirements and design an appropriate software architecture by applying software engineering principles. Your submission should demonstrate your ability to select, justify, and evaluate a suitable architectural style.

Question

Based on your **assigned problem statement**, perform an **Architecture Design Analysis** and prepare an architecture design report by answering the following questions:

1. Analyze System Requirements

- Study the given problem statement and identify the system objectives.
- Analyze the functional and non-functional requirements that influence the software architecture.
- Identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.

2. Select a Suitable Software Architecture

- Select an appropriate architectural style for the proposed system (e.g., Layered Architecture, Client-Server, MVC, Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture).
- Justify why the selected architecture is the most suitable for the given problem.

3. Draw the Software Architecture Diagram

- Design a clear architecture diagram illustrating the overall structure of the proposed system.
- Include all major layers/components, databases, external systems, APIs, and communication mechanisms.
- Use appropriate software architecture notations and labels.

4. **Explain Components and Their Interactions**

- Describe the purpose and responsibilities of each architectural component.
- Explain how the components communicate and exchange data.
- Illustrate the overall workflow of the system.

5. **Discuss Quality Attributes**

Evaluate how the proposed architecture addresses the following aspects:

- Scalability
- Security
- Performance
- Reliability
- Maintainability
- Availability

6. **Justify Architectural Decisions**

- Explain the rationale behind your architectural choices.
- Discuss the trade-offs considered while selecting the architecture.
- Explain how the chosen architecture supports future enhancements, system integration, and long-term maintainability.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis (30%)	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture (25%)	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.
Architecture Diagram and Component Design (20%)	Produces a clear, complete, and well-structured architecture diagram with all major components,	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and lacks sufficient	Diagram is incomplete, unclear, or technically incorrect.

	interfaces, and interactions accurately represented.		detail.	
Component Interaction and Quality Attribute Analysis (15%)	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.
Architectural Justification and Technical Presentation (10%)	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-organized, professional report.	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope; report needs improvement.	Justification is weak or absent; report lacks organization and technical clarity.

Criteria	**Mark**
System Requirement Analysis **(30%)**	/5
Selection of Software Architecture **(25%)**	/5
Architecture Diagram and Component Design **(20%)**	/5
Component Interaction and Quality Attribute Analysis **(15%)**	/5
Architectural Justification and Technical Presentation **(10%)**	/5
TOTAL	/25

1. System Requirements Analysis

1.1 System Objectives

- Create IT service tickets for employee-reported technical problems.
- Categorize technical problems so that tickets can be routed to the appropriate service area.
- Prioritize incidents using impact and urgency.
- Assign tickets to suitable IT support teams or support staff.
- Track ticket status from creation through investigation, resolution and closure.
- Monitor service-level agreements and identify tickets approaching or exceeding targets.
- Maintain a searchable technical knowledge base for common problems and solutions.
- Collect user feedback after a ticket is resolved.
- Generate analytics to measure workload, resolution performance and SLA compliance.

1.2 Functional Requirements

ID	Requirement	Architectural Impact
FR-01	User authentication and role management.	Central authentication and role-based authorization are required.
FR-02	Employee creates and submits a service ticket.	Ticket API and persistent ticket storage are required.
FR-03	System categorizes the technical issue.	Categorization rules must be available in the ticket-processing module.
FR-04	System assigns incident priority.	Priority rules must use impact and urgency information.
FR-05	Ticket is assigned to a support team/staff member.	Assignment logic and assignment history are required.
FR-06	Users can view and update ticket status according to role.	Ticket lifecycle and status-transition rules are required.
FR-07	System monitors SLA response and resolution targets.	Timestamp storage and SLA monitoring logic are required.
FR-08	Users can search the technical knowledge base.	Searchable knowledge storage and API are required.
FR-09	Authorized staff can maintain knowledge articles.	Role-controlled article creation, editing and approval are required.
FR-10	System collects user feedback.	Feedback records must be linked to resolved tickets.
FR-11	System generates service analytics.	Structured ticket, SLA and feedback data must support reporting.
FR-12	System sends important notifications.	Notification interface is required for assignment, updates and SLA alerts.
FR-13	System maintains activity/audit history.	Important actions must be recorded with timestamps and users.

1.3 Non-Functional Requirements

ID	Quality Requirement	Target / Design Consideration
NFR-01	Scalability	Application/API layer should support additional instances when user or ticket volume increases.
NFR-02	Security	Use authentication, role-based authorization, HTTPS, validation and protected database access.
NFR-03	Performance	Use indexed fields, efficient queries and lightweight API responses.
NFR-04	Reliability	Use transactional updates and validation for critical ticket operations.
NFR-05	Availability	Keep application modules separated and support backup/recovery procedures.
NFR-06	Maintainability	Use clear layers, modular services, interfaces and readable data-access logic.
NFR-07	Usability	Provide simple role-based screens with clear ticket status and actions.
NFR-08	Auditability	Record important status, assignment, resolution and administrative changes.

1.4 Key Quality Attributes

The architecture is mainly driven by scalability, security, performance, reliability, maintainability and availability. Maintainability is important because support teams, ticket categories and SLA rules may change over time. Security is also important because tickets can contain internal organizational information. These quality attributes therefore influence the separation of layers, the use of role-based access and the choice of a consistent relational data store.

2. Selection of Software Architecture

2.1 Selected Architecture – Hybrid Layered Architecture

A Hybrid Layered Architecture is selected for the proposed system. The architecture uses a clear separation between the presentation layer, application/API layer and data layer. Inside the application layer, related business functions are organized into modular services for ticket management, SLA and analytics, and knowledge and feedback. This gives the project clear boundaries without introducing the operational complexity of a full microservices system.

2.2 Architectural Layers

- Presentation Layer – provides web pages and dashboards for employees, support staff, IT managers and administrators.
- Application/API Layer – handles authentication, authorization, validation, API requests, workflow and notifications.
- Business Service Modules – contain ticket management, SLA monitoring, analytics, knowledge-base and feedback logic.
- Data Layer – stores users, tickets, assignments, SLA data, knowledge articles, feedback and audit information.
- External Integration Layer – connects the application to email or other notification services.

2.3 Reasons for Selection

Reason	Justification
Appropriate system size	The system has several related functions but does not initially require many independently deployed services.
Clear separation	Presentation, business rules and data access remain separated, reducing coupling.
Maintainability	Ticket, SLA, knowledge and analytics functions can be developed and tested as separate modules.
Scalability	The API/application layer can later be replicated or selected modules can be extracted when required.
Security	Authentication and authorization can be enforced at a common application boundary.
Integration	REST APIs provide a straightforward way to connect web clients and future external systems.
Implementation feasibility	A student team can implement the design as a modular application without maintaining a complex distributed infrastructure.

2.4 Alternatives Considered

Architecture	Decision	Reason
Pure Layered Architecture	Considered	Simple and suitable, but the hybrid design gives clearer boundaries for major service functions.
Microservices	Not selected initially	Would add service deployment, monitoring, communication and distributed-data complexity.
Event-Driven Architecture	Partially used	Useful for notifications and SLA events, but unnecessary as the architecture for every operation.
MVC	Partially applicable	Useful within the web implementation, but it does not represent the complete system architecture.
Serverless	Not selected	The initial system benefits from predictable transactional processing and a straightforward database model.

3. Software Architecture Diagram

The proposed architecture below shows the main users, application layers, modular services, persistent data stores and notification integration. Arrows indicate the main communication paths.

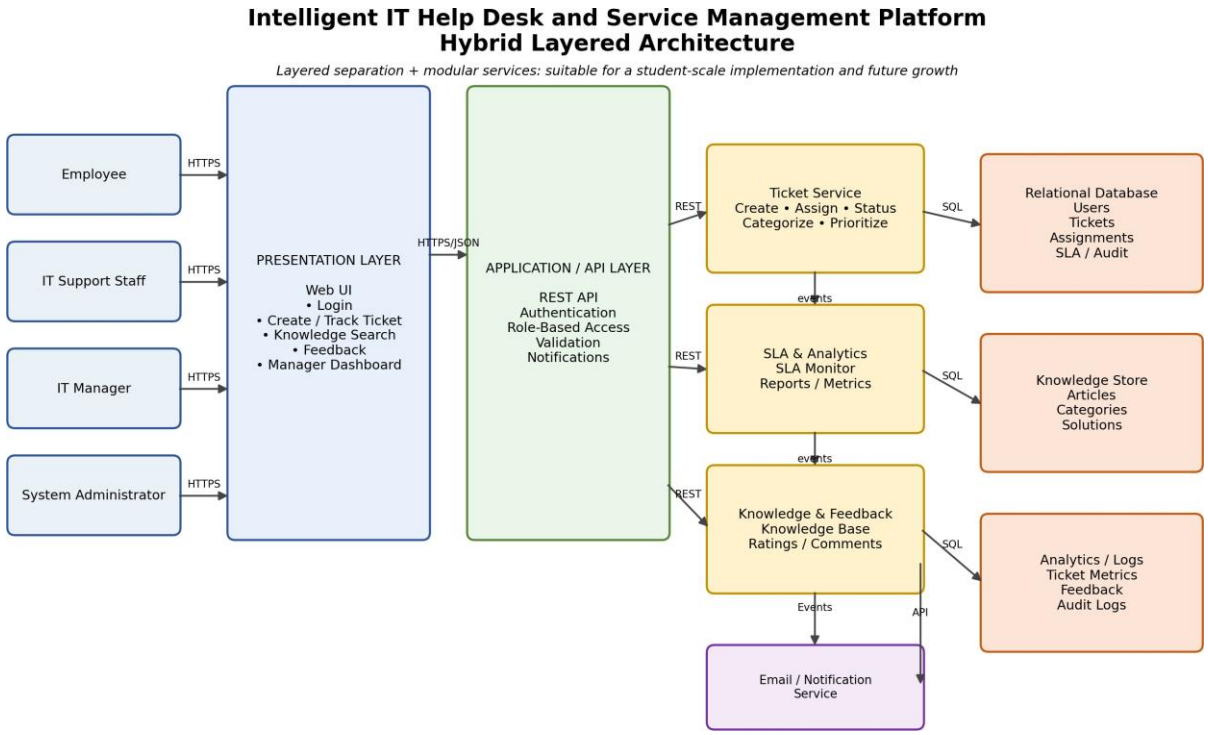


Figure 1. Hybrid Layered Architecture for the Intelligent IT Help Desk and Service Management Platform

3.1 Communication Paths

- Users access the web interface through HTTPS.
- The presentation layer communicates with the application/API layer using HTTPS and JSON-based requests.
- The application layer invokes the appropriate business service for each operation.
- Business services read or update persistent data through controlled database operations.
- Important events can invoke the notification service for email or in-app alerts.
- Analytics and audit information is generated from structured ticket, SLA and feedback records.

4. Component Responsibilities and Interactions

4.1 Component Responsibilities

Component	Responsibility
Web UI / Presentation	Provides login, ticket creation, ticket tracking, knowledge search, feedback and dashboard screens.
Authentication & Authorization	Validates user identity and checks permissions for protected operations.
Ticket Service	Creates tickets, applies categories and priority, manages assignment and controls status transitions.
SLA & Analytics Service	Calculates SLA status and prepares service-performance metrics and reports.
Knowledge & Feedback Service	Manages searchable knowledge articles and records employee feedback.
Notification Service	Triggers messages for assignment, status changes, resolution and important SLA events.
Relational Database	Stores transactional users, tickets, assignments, SLA, feedback and audit records.
Knowledge Store	Stores knowledge articles, categories and reusable solutions.
Analytics / Audit Data	Supports reports, feedback analysis and traceability of important actions.

4.2 Component Interaction

An employee first authenticates through the Web UI. When the employee creates a ticket, the request is sent to the API layer. The application validates the request and passes it to the Ticket Service. The Ticket Service creates the ticket, assigns category and priority according to the configured rules, and stores the record in the relational database.

Support staff retrieve assigned tickets through the same API. Status changes, assignment changes and resolution details are validated and stored with timestamps. The SLA component checks response and resolution targets using the stored

timestamps. If a ticket approaches or exceeds an SLA threshold, the notification service can send an alert.

After the ticket is resolved, the employee can provide feedback. The knowledge service allows users to search common solutions and authorized staff to maintain articles. Managers can view analytics such as ticket volume, average resolution time, category distribution and SLA compliance.

4.3 End-to-End Workflow

- 1. Employee logs into the platform.
- 2. Employee creates an IT service ticket.
- 3. API validates the ticket information.
- 4. Ticket Service assigns category and priority.
- 5. Ticket is assigned to a suitable support team or staff member.
- 6. Support staff investigate and update the ticket.
- 7. SLA Service checks response and resolution targets.
- 8. Notifications are sent for important ticket events when required.
- 9. Support staff record the resolution and close the ticket.
- 10. Employee submits feedback.
- 11. Ticket, SLA and feedback information contributes to service analytics.

5. Quality Attribute Analysis

5.1 Scalability

The separation between presentation, application and data layers allows the application/API layer to be scaled independently. If ticket volume increases, multiple application instances can be deployed behind a load balancer. A heavily used module such as analytics or notifications can later be extracted into an independent service. Database indexes and optimized queries support increasing ticket volumes.

5.2 Security

All protected operations pass through authentication and role-based authorization. Employee, support staff, manager and administrator permissions are separated. HTTPS protects data during transmission, input validation reduces malformed requests, and the database is accessed through the application rather than directly from the browser. Audit records provide traceability for important changes.

5.3 Performance

The architecture keeps operations focused on the required module. Frequently searched ticket fields can be indexed, API responses can be limited to necessary information, and heavy analytics queries can be separated from normal ticket transactions. This prevents reporting operations from unnecessarily slowing everyday ticket handling.

5.4 Reliability

Critical operations such as ticket creation, assignment and status updates should use transactional database operations. Validation prevents incomplete ticket states. Audit records preserve a history of important changes. Notification failure should not remove or invalidate a ticket because the database remains the primary source of truth.

5.5 Maintainability

The architecture separates presentation, business logic and data access. The modular service structure also keeps ticket management, SLA and analytics, and knowledge and feedback responsibilities distinct. This allows changes to one business function without requiring changes throughout the entire system.

5.6 Availability

The application can later be deployed with multiple application instances and database backup/recovery procedures. Since users communicate through the API layer, an application instance can be replaced without changing the user interface. This supports easier maintenance and recovery.

6. Architectural Justification and Trade-Offs

6.1 Main Architectural Decision

The hybrid layered architecture provides the best balance between functionality, maintainability and implementation effort. A full microservices approach would be technically possible, but it would introduce additional infrastructure and operational responsibilities that are not justified for the initial system. The selected design provides modularity now while keeping a clear path for future service extraction.

6.2 Major Trade-Offs

Decision	Advantage	Trade-Off
Layered separation	Easy understanding, testing and maintenance.	Requests pass through additional layers.
Modular services	Clear responsibilities and easier future extraction.	Interfaces between modules must be maintained.
Relational database	Strong consistency for ticket and assignment data.	Very large analytics workloads may eventually need a separate reporting store.
REST API	Simple web and external-system integration.	API versioning and authentication need continued management.
Notification events	Useful for alerts and SLA monitoring.	Asynchronous processing adds some implementation complexity.

6.3 Future Enhancements

- Integrate single sign-on or organizational directory services.
- Add machine-learning based ticket category suggestions.
- Recommend relevant knowledge articles while the employee enters a ticket.
- Introduce a message broker if notification and SLA workloads become large.
- Extract high-load modules into independently deployed services when justified by real usage.
- Add a mobile-friendly interface or mobile application.
- Integrate asset monitoring so device or network alerts can automatically create tickets.

6.4 Long-Term Maintainability

The design intentionally avoids over-engineering the first version. The API boundary and modular service structure create a migration path. The initial system can be deployed as one application, while a module can later be separated if its workload, reliability or business importance requires independent scaling. This provides a practical balance between present implementation needs and future growth.

7. Conclusion

The proposed Hybrid Layered Architecture is suitable for the Intelligent IT Help Desk and Service Management Platform because it directly supports the required ticket lifecycle, SLA monitoring, knowledge management, feedback and analytics functions. The architecture separates the user interface, application/API logic, business modules and persistent data, which improves maintainability and security.

The design also addresses the required quality attributes of scalability, security, performance, reliability, maintainability and availability. It remains realistic for an initial student implementation while providing a clear path for future enhancements such as machine-learning assistance, single sign-on, asset integration and selective migration to independent services.

8. Implementation Requirement

For this Architecture Design Assignment, a full software implementation is not required. The required submission is the architecture analysis and design: system requirements, architecture selection and justification, architecture diagram, component responsibilities and interactions, quality-attribute analysis, and architectural decisions/trade-offs.

A prototype can be developed later if the faculty separately asks for implementation. If implemented, the architecture can be realized using a web front end, REST API backend and relational database while keeping the same logical layers and service boundaries.